

ACARA 21. Middleware

Pokok Bahasan : **Middleware**
Acara Praktikum : **Minggu 6/1**
Tempat : **Daring/Luring Politeknik Negeri Jember**
Alokasi Waktu : **100 Menit**

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep dasar middleware dalam laravel.
2. Mahasiswa mampu mengimplementasikan middleware dalam aplikasi laravel.
3. Mahasiswa mampu menerapkan middleware dengan parameter.
4. Mahasiswa mampu menjelaskan siklus hidup middleware.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep dasar middleware dalam laravel.
2. Mahasiswa mengimplementasikan middleware dalam aplikasi laravel.
3. Mahasiswa menerapkan middleware dengan parameter.
4. Mahasiswa menjelaskan siklus hidup middleware.

d. Dasar Teori

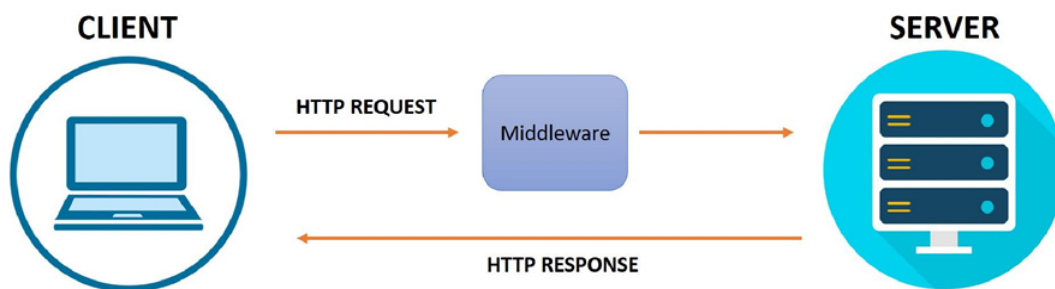
Apa Itu Middleware di Laravel?, Middleware di Laravel adalah lapisan perantara (interceptor) yang menangani permintaan HTTP sebelum mencapai controller atau setelah respons dikirim ke klien. Middleware digunakan untuk memproses logika tertentu seperti autentikasi, logging, CORS, validasi token, dan lainnya.

Middleware bisa digunakan untuk berbagai tujuan, seperti:

- a) Memeriksa apakah pengguna telah login.
- b) Mengecek peran atau izin pengguna.
- c) Memfilter atau memodifikasi permintaan sebelum mencapai controller.
- d) Menangani CORS (Cross-Origin Resource Sharing).
- e) Melakukan logging terhadap permintaan HTTP.

2. Cara Kerja Middleware

Middleware bekerja seperti rantai (pipeline) yang mengelola request dan response. Berikut adalah alur kerjanya:



- a) Client mengirimkan permintaan HTTP ke server.
- b) Middleware menangkap dan memproses permintaan.
- c) Jika request valid, diteruskan ke controller.
- d) Controller memproses request dan mengembalikan response.
- e) Middleware kembali menangkap response sebelum dikirim ke client.
- f) Client menerima response dari server.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 10
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Kertas A4 / Folio Bergaris
6. Pulpen

f. Prosedur Kerja

1) Membuat Middleware di Laravel

Untuk membuat middleware baru, gunakan perintah Artisan:

```
php artisan make:middleware Admin
```

Perintah ini akan membuat file `Admin.php` di direktori `app/Http/Middleware`.

2) Struktur Middleware

Middleware yang baru dibuat akan terlihat seperti ini:

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
use Auth;

class Admin {
    public function handle(Request $request, Closure $next){
        if (Auth::check()){
            if(Auth::user()->role == 'admin'){
                return $next($request);
            } else {
                abort(403);
            }
        } else {
            return redirect()->route('login');
        }
    }
}
```

- `$request->age` mengambil parameter `age` dari request.
- Jika umur kurang dari 18 tahun, permintaan ditolak dengan status `403 Forbidden`.
- Jika memenuhi syarat, request diteruskan ke controller dengan `$next($request)`.

3) Mendaftarkan Middleware

Setelah middleware dibuat, pada Laravel 10 kita harus mendaftarkannya di file `app/Http/Kernel.php`. Sedangkan Pada Laravel 11 & 12 kita harus mendaftarkannya di file `bootstrap/app.php`.

a) Middleware Global

Jika ingin middleware berlaku untuk semua request, tambahkan di bagian `$middleware` :

```
protected $middleware = [  
    \App\Http\Middleware\Admin::class,  
];
```

b) Middleware Grup

Jika middleware hanya digunakan untuk grup tertentu (misalnya API atau web), tambahkan di bagian `$middlewareGroups`:

```
protected $middlewareGroups = [  
    'web' => [  
        \App\Http\Middleware\Admin::class,  
    ],  
    'api' => [  
        \App\Http\Middleware\Admin::class,  
    ],  
];
```

c) Middleware Route / Alias

Untuk middleware yang hanya digunakan pada route tertentu, daftarkan di `$routeMiddleware`:

```
protected $routeMiddleware = [  
    'admin' => \App\Http\Middleware\Admin::class,  
];
```

pada Laravel 11 & 12 middleware didaftarkan di `->withMiddleware()`

```
$middleware->alias([  
    'admin' => Admin::class,  
    'petugas' => Petugas::class,  
    'siswa' => Siswa::class,  
]);
```

4) Menggunakan Middleware pada Route

Setelah middleware terdaftar, kita bisa menggunakannya di route:

```
Route::get('/admin', function () {  
    return "Selamat datang!";  
})->middleware('admin');
```

5) Middleware dengan Parameter

Middleware juga bisa menerima parameter, misalnya untuk membatasi akses berdasarkan peran:

a) Membuat Middleware

```
public function handle(Request $request, Closure $next, $role)  
{  
    if ($request->user()->role !== $role) {  
        return response()->json(['message' => 'Akses ditolak!'], 403);  
    }  
    return $next($request);  
}
```

b) Menggunakannya di Route

```
Route::get('/admin', function () {  
    return "Halaman Admin";  
})->middleware('cek.role:admin');
```

6) Middleware Terminate

Middleware bisa memiliki metode `terminate()` untuk menangani tugas setelah respons dikirim ke klien. Contohnya untuk logging:

```
public function terminate($request, $response)  
{  
    \Log::info('Request selesai', ['url' => $request->fullUrl()]);  
}
```

Untuk menggunakan fitur ini, pastikan middleware terdaftar di `Kernel.php`.

g. Hasil dan Pembahasan**Format Laporan**

- Kertas A4
- Format *.pdf
- Struktur Laporan: Cover, Pendahuluan, Hasil Praktik, Kesimpulan dan Daftar Pustaka
- Berikan Identitas Diri: NIM, Nama, Golongan, Tugas Minggu Ke-X Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-21_A_E1234356_BUDI.pdf
- Kumpulkan pada <https://elearning-jti.polije.ac.id/>

h. Kesimpulan

Dengan memahami konsep dasar middleware dalam laravel, mengimplementasikan middleware dalam aplikasi laravel, menerapkan middleware dengan parameter, menjelaskan siklus hidup middleware, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

ACARA 22. Authentication

Pokok Bahasan	: Authentication (Part 1)
Acara Praktikum	: Minggu 6/2
Tempat	: Daring/Luring Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami memahami konsep autentikasi di laravel.
2. Mahasiswa mampu menerapkan Laravel Breeze sebagai starter kit autentikasi.
3. Mahasiswa mampu menerapkan middleware route dan halaman Aplikasi.
4. Mahasiswa mampu mengimplementasikan fitur verifikasi email.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami memahami konsep autentikasi di laravel.
2. Mahasiswa menerapkan Laravel Breeze sebagai starter kit autentikasi.
3. Mahasiswa menerapkan middleware route dan halaman Aplikasi.
4. Mahasiswa mengimplementasikan fitur verifikasi email.

d. Dasar Teori

Laravel Authentication adalah fitur bawaan Laravel yang digunakan untuk mengelola proses autentikasi pengguna secara aman dan efisien. Laravel menyediakan sistem autentikasi yang fleksibel dan mudah dikonfigurasi, mencakup login, registrasi, reset password, dan manajemen sesi pengguna.

Dengan berbagai pilihan seperti Laravel Breeze, Jetstream, dan Sanctum, pengembang dapat menyesuaikan autentikasi sesuai dengan kebutuhan proyek, baik itu aplikasi berbasis web maupun API. Jika Anda ingin membangun sistem autentikasi Laravel dengan cepat, gunakan Breeze atau Jetstream. Namun, jika Anda ingin mengembangkan solusi khusus, gunakan pendekatan manual dengan Auth Facade atau Sanctum untuk API.

1. Komponen Utama Laravel Authentication

Laravel Authentication terdiri dari beberapa komponen utama:

a) Guards

Guards adalah mekanisme yang digunakan untuk menentukan bagaimana pengguna diautentikasi. Laravel menyediakan beberapa jenis guard:

- 1) **Session Guard** → Menggunakan sesi dan cookies untuk autentikasi (default).
- 2) **Token Guard** → Menggunakan token API untuk autentikasi (cocok untuk aplikasi RESTful).
- 3) **Sanctum & Passport** → Digunakan untuk autentikasi API yang lebih kompleks.

Guards dikonfigurasi di dalam file **config/auth.php**:

```
'guards' => [  
    'web' => [  
        'driver' => 'session',  
        'provider' => 'users',  
    ],  
    'api' => [  
        'driver' => 'token',  
        'provider' => 'users',  
        'hash' => false,  
    ],  
],
```

b) Providers

Providers menentukan bagaimana pengguna diambil dari database. Laravel secara default menggunakan **Eloquent ORM** atau **Query Builder**. Contoh konfigurasi provider di **config/auth.php**:

```
'providers' => [  
    'users' => [  
        'driver' => 'eloquent',  
        'model' => App\Models\User::class,  
    ],  
],
```

Anda juga bisa menggunakan query builder:

```
'users' => [  
    'driver' => 'database',  
    'table' => 'users',  
],
```

e. Alat dan Bahan

1. PC / Laptop
2. Windows 10
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Text Editor : Visual Studio Code
6. Kertas A4 / Folio Bergaris
7. Pulpen

f. Prosedur Kerja

1. Cara Menggunakan Laravel Authentication

Laravel menyediakan berbagai cara untuk mengimplementasikan autentikasi, mulai dari penggunaan **starter kit** hingga pembuatan kustom.

a) Menggunakan Laravel Breeze (Starter Kit Sederhana)

Laravel Breeze adalah starter kit autentikasi yang ringan dan mudah dikonfigurasi.

Langkah-langkah instalasi:

Instal Laravel:

```
composer create-project laravel/laravel nama-proyek
```

Instal Laravel Breeze:

```
composer require laravel/breeze --dev
```

Jalankan perintah instalasi:

```
php artisan breeze:install
```

Jalankan migrasi:

```
php artisan migrate
```

Jalankan server:

```
php artisan serve
```

Setelah itu, Laravel akan menyediakan tampilan login, registrasi, dan reset password secara otomatis.

b) Menggunakan Laravel Jetstream (Starter Kit Lanjutan)

Laravel Jetstream adalah versi lebih canggih dari Breeze dengan fitur tambahan seperti **two-factor authentication (2FA)**, **team management**, dan **API token authentication**. Langkah-langkah instalasi Jetstream:

Instal Jetstream:

```
composer require laravel/jetstream
```

Jalankan instalasi:

```
php artisan jetstream:install livewire
```

Atau jika menggunakan Inertia.js:

```
php artisan jetstream:install inertia
```

Jalankan migrasi:

```
php artisan migrate
```

c) Menggunakan Manual Authentication (Tanpa Starter Kit)

Jika Anda ingin mengimplementasikan autentikasi secara manual, Anda dapat menggunakan fitur **Auth Facade**.

Login Manual

```
use Illuminate\Support\Facades\Auth;  
use Illuminate\Http\Request;  
  
public function login(Request $request) {  
    $credentials = $request->only('email', 'password');  
}
```

```
if (Auth::attempt($credentials)) {  
    return redirect()->intended('dashboard');  
}  
  
return back()->withErrors(['email' => 'Email atau password salah.']);  
}
```

Logout

```
public function logout(Request $request) {  
    Auth::logout();  
    $request->session()->invalidate();  
    return redirect('/');  
}
```

Registrasi

```
use App\Models\User;  
use Illuminate\Support\Facades\Hash;  
  
public function register(Request $request) {  
    $user = User::create([  
        'name' => $request->name,  
        'email' => $request->email,  
        'password' => Hash::make($request->password),  
    ]);  
  
    Auth::login($user);  
  
    return redirect('dashboard');  
}
```

2. API Authentication dengan Laravel Sanctum

Untuk autentikasi API yang aman dan ringan, Laravel menyediakan **Sanctum**.

Instalasi Laravel Sanctum

```
composer require laravel/sanctum
```

Kemudian, jalankan perintah:

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"  
php artisan migrate
```

Tambahkan middleware Sanctum di **app/Http/Kernel.php**:

```
'api' => [  
    \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,  
    'throttle:api',  
    \Illuminate\Routing\Middleware\SubstituteBindings::class,  
],
```

3. Menggunakan Token API

Membuat Token API

```
$token = $user->createToken('token-name')->plainTextToken;
```

Menggunakan Middleware Sanctum Di dalam routes/api.php:

```
Route::middleware('auth:sanctum')->get('/user', function (Request $request) {  
    return $request->user();  
});
```

Login dengan Token

```
public function login(Request $request) {  
    $user = User::where('email', $request->email)->first();  
  
    if (!$user || !Hash::check($request->password, $user->password)) {  
        return response()->json(['message' => 'Unauthorized'], 401);  
    }  
  
    return response()->json([  
        'token' => $user->createToken('authToken')->plainTextToken,  
    ]);  
}
```

4. Middleware Authentication

Middleware digunakan untuk melindungi rute-rute tertentu agar hanya pengguna yang sudah login yang bisa mengaksesnya.

Contoh penggunaan middleware:

```
Route::middleware(['auth'])->group(function () {  
    Route::get('/dashboard', function () {  
        return view('dashboard');  
    });  
});
```

Untuk API:

```
Route::middleware(['auth:sanctum'])->get('/profile',  
    function (Request $request) {  
        return $request->user();  
    });
```

5. Reset Password dan Verifikasi Email

Reset Password

Laravel sudah menyediakan fitur reset password bawaan. Cukup jalankan perintah:

```
php artisan make:auth
```

Lalu, Laravel akan otomatis menghasilkan sistem reset password.

Verifikasi Email

Aktifkan fitur verifikasi email di model **User.php**:

```
use Illuminate\Contracts\Auth\MustVerifyEmail;  
class User extends Authenticatable implements MustVerifyEmail  
{  
    //  
}
```

Tambahkan middleware 'verified' di **routes/web.php**:

```
Route::get('/dashboard', function () {  
    return view('dashboard');  
})->middleware(['auth', 'verified']);
```

Tambahkan Mengirim Email Verifikasi

Setelah registrasi, Laravel otomatis mengirim email verifikasi. Jika ingin mengirim ulang, tambahkan rute berikut:

```
use Illuminate\Foundation\Auth\EmailVerificationRequest;  
  
Route::get('/email/verify', function () {  
    return view('auth.verify-email');  
})->middleware('auth')->name('verification.notice');
```

```
Route::get('/email/verify/{id}/{hash}', function (EmailVerificationRequest $request) {  
    $request->fulfill();  
    return redirect('/dashboard');  
})->middleware(['auth', 'signed'])->name('verification.verify');  
  
Route::post('/email/resend', function (Request $request) {  
    $request->user()->sendEmailVerificationNotification();  
    return back()->with('message', 'Verification link sent!');  
})->middleware(['auth', 'throttle:6,1'])->name('verification.resend');
```

Menambahkan Link Verifikasi di Blade

Di **resources/views/auth/verify-email.blade.php**:

```
@if (session('message'))  
    <p>{{ session('message') }}</p>  
@endif  
  
<form method="POST" action="{{ route('verification.resend') }}">  
    @csrf  
    <button type="submit">Resend Verification Email</button>  
</form>
```

Jika pengguna belum memverifikasi email, mereka akan diarahkan ke halaman ini.

g. Hasil dan Pembahasan

Format Laporan

- Kertas A4
- Format *.pdf
- Struktur Laporan: Cover, Pendahuluan, Hasil Praktik, Kesimpulan dan Daftar Pustaka
- Berikan Identitas Diri: NIM, Nama, Golongan, Tugas Minggu Ke-X Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-22_A_E1234356_BUDI.pdf
- Kumpulkan pada <https://elearning-jti.poliije.ac.id/>

h. Kesimpulan

Dengan memahami memahami konsep autentikasi di laravel menerapkan Laravel Breeze sebagai starter kit autentikasi, menerapkan middleware route dan halaman aplikasi, mengimplementasikan fitur verifikasi email, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

ACARA 23. Authentication

Pokok Bahasan	: Authentication (Part 2)
Acara Praktikum	: Minggu 6/3
Tempat	: Daring/Luring Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep autentikasi di laravel.
2. Mahasiswa mampu menerapkan Laravel Breeze sebagai starter kit autentikasi.
3. Mahasiswa mampu menerapkan middleware route dan halaman Aplikasi.
4. Mahasiswa mampu mengimplementasikan fitur verifikasi email.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep autentikasi di laravel.
2. Mahasiswa menerapkan Laravel Breeze sebagai starter kit autentikasi.
3. Mahasiswa menerapkan middleware route dan halaman Aplikasi.
4. Mahasiswa mengimplementasikan fitur verifikasi email.

d. Dasar Teori

Dasar teori untuk Acara 23 mengikuti dasar teori pada Acara 22 poin d.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 10
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Composer
6. Visual Studio Code
7. Kertas A4 / Folio Bergaris
8. Pulpen

f. Prosedur Kerja

1. Redirect User To Specific Page

Setelah pengguna berhasil login atau register, kita dapat mengarahkan mereka ke halaman tertentu.

Mengubah Redirect Default

Laravel menangani redirect setelah login di file `app/Providers/RouteServiceProvider.php`.

Ubah properti `$HOME`:

```
public const HOME = '/dashboard';
```

Dengan ini, setelah login, pengguna akan diarahkan ke `/dashboard`.

2. Retrieving The Authenticated User

Untuk mendapatkan data pengguna yang sedang login, kita bisa menggunakan **Auth Facade**. Mengambil Data Pengguna di Controller

```
use Illuminate\Support\Facades\Auth;
public function showProfile() {
    $user = Auth::user(); // Mendapatkan user yang sedang login
    return view('profile', compact('user'));
}
```

Menggunakan di Blade Template

Di dalam file `profile.blade.php`:

```
<h1>Welcome, {{ Auth::user()->name }}</h1>
```

Mengambil ID Pengguna

```
$userId = Auth::id();
```

Jika pengguna belum login, metode ini akan mengembalikan `null`.

3. Recreating Logout Feature

Untuk membuat fitur logout secara manual, kita bisa menggunakan **Auth Facade**.

a) Menambahkan Logout di Controller

Buka atau buat file **AuthController.php**:

```
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class AuthController extends Controller {
    public function logout(Request $request) {
        Auth::logout(); // Logout user

        $request->session()->invalidate(); // Hapus sesi
        $request->session()->regenerateToken();
        // Regenerasi CSRF token

        return redirect('/'); // Redirect ke halaman utama
    }
}
```

b) Menambahkan Rute Logout

Tambahkan rute di **routes/web.php**:

```
Route::post('/logout', [AuthController::class, 'logout'])->name('logout');
```

c) Menambahkan Tombol Logout di Blade

Tambahkan di **layouts/app.blade.php**:

```
<form action="{{ route('logout') }}" method="POST">
    @csrf
    <button type="submit">Logout</button>
</form>
```

Dengan ini, pengguna dapat logout dengan aman.

4. Protecting Routes

Kita dapat melindungi rute agar hanya bisa diakses oleh pengguna yang telah login.

a) Menggunakan Middleware `auth`

Di **routes/web.php**, bungkus rute dengan middleware `auth`:

```
Route::middleware(['auth'])->group(function () {  
    Route::get('/dashboard', function () {  
        return view('dashboard');  
    });  
});
```

Jika pengguna belum login, mereka akan diarahkan ke halaman login.

b) Melindungi API Routes

Di **routes/api.php**, gunakan middleware `auth:sanctum`:

```
Route::middleware('auth:sanctum')->get('/user', function (Request  
$request) {  
    return $request->user();  
});
```

c) Melindungi Konten di Blade

Gunakan directive `@auth` dan `@guest`:

```
@auth  
    <p>Selamat datang, {{ Auth::user()->name }}</p>  
@endauth  
  
@guest  
    <p>Silakan login untuk mengakses fitur ini.</p>  
@endguest
```

g. Hasil dan Pembahasan**Format Laporan**

- Kertas A4
- Format *.pdf
- Struktur Laporan: Cover, Pendahuluan, Hasil Praktik, Kesimpulan dan Daftar Pustaka
- Berikan Identitas Diri: NIM, Nama, Golongan, Tugas Minggu Ke-X Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-23_A_E1234356_BUDI.pdf
- Kumpulkan pada <https://elearning-jti.polije.ac.id/>

h. Kesimpulan

Dengan memahami konsep dasar Eloquent ORM, menerapkan operasi CRUD (Create, Read, Update, Delete) menggunakan Eloquent, menggunakan conditional clause dalam Query Eloquent, mengelola hubungan antar tabel menggunakan Eloquent Relationships, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

ACARA 24. Authorization

Pokok Bahasan	: Authorization
Acara Praktikum	: Minggu 6/4
Tempat	: Daring/Luring Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep authorization dalam laravel.
2. Mahasiswa mampu menerapkan gates untuk kontrol akses.
3. Mahasiswa mampu mengimplementasikan policies dalam laravel.
4. Mahasiswa mampu menggunakan middleware authorization dalam routing.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep authorization dalam laravel.
2. Mahasiswa menerapkan gates untuk kontrol akses.
3. Mahasiswa mengimplementasikan policies dalam laravel.
4. Mahasiswa menggunakan middleware authorization dalam routing.

d. Dasar Teori

Laravel menyediakan mekanisme authorization yang fleksibel untuk mengontrol akses pengguna ke berbagai bagian aplikasi. Otorisasi ini digunakan untuk memastikan bahwa hanya pengguna yang memiliki hak tertentu yang dapat melakukan suatu tindakan. Dalam Laravel, authorization dapat dikelola dengan gates dan policies.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 10
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Composer
6. Visual Studio Code
7. Kertas A4 / Folio Bergaris
8. Pulpen

f. Prosedur Kerja

1. Gates (Gerbang)

Gates adalah fungsi sederhana yang menentukan apakah pengguna memiliki izin untuk melakukan tindakan tertentu. Gates biasanya didefinisikan dalam `App\Providers\AuthServiceProvider`.

a) Cara Menggunakan Gates & Menentukan Gate

Tambahkan dalam `AuthServiceProvider.php`:

```
use Illuminate\Support\Facades\Gate;
use App\Models\User;

public function boot()
{
    Gate::define('edit-post', function (User $user, $post) {
        return $user->id === $post->user_id;
    });
}
```

b) Menggunakan Gate dalam Controller

```
use Illuminate\Support\Facades\Gate;

public function update(Request $request, Post $post)
{
    if (Gate::denies('edit-post', $post)) {
        abort(403, 'Anda tidak memiliki izin untuk mengedit postingan ini.');
```

c) Menggunakan Gate dalam Blade

```
@can('edit-post', $post)
    <button>Edit</button>
@endcan
```

2. Policies (Kebijakan)

Policies adalah kelas khusus yang menangani logika otorisasi untuk model tertentu. Biasanya digunakan untuk skenario yang lebih kompleks dibandingkan **gates**.

a) Membuat dan Menggunakan Policies

untuk Membuat Policy Jalankan perintah artisan:

```
php artisan make:policy PostPolicy
```

Ini akan menghasilkan file policy di `app/Policies/PostPolicy.php`.

b) Mendefinisikan Otorisasi dalam Policy

Buka `PostPolicy.php` dan tambahkan metode otorisasi:

```
namespace App\Policies;

use App\Models\User;
use App\Models\Post;

class PostPolicy
{
    public function update(User $user, Post $post)
    {
        return $user->id === $post->user_id;
    }
}
```

c) Mendaftarkan Policy di `AuthServiceProvider`

```
use App\Models\Post;
use App\Policies\PostPolicy;

protected $policies = [
    Post::class => PostPolicy::class,
];
```

d) Menggunakan Policy dalam Controller

```
public function update(Request $request, Post $post)
{
    $this->authorize('update', $post);
    // Jika lolos, lanjutkan update
}
```

e) Menggunakan Policy dalam Blade

```
@can('update', $post)
    <button>Edit</button>
@endcan
```

3. Middleware Authorization

Selain menggunakan **gates** dan **policies**, Laravel juga menyediakan middleware **can** untuk mengontrol akses dalam route.

a) Menentukan Middleware dalam Route

```
Route::get('/post/{post}/edit', [PostController::class, 'edit'])
    ->middleware('can:update,post');
```

b) Middleware dalam Controller

```
public function __construct()
{
    $this->middleware('can:update,post')->only('edit', 'update');
}
```

g. Hasil dan Pembahasan

Format Laporan

- Kertas A4
- Format *.pdf
- Struktur Laporan: Cover, Pendahuluan, Hasil Praktik, Kesimpulan dan Daftar Pustaka
- Berikan Identitas Diri: NIM, Nama, Golongan, Tugas Minggu Ke-X Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-24_A_E1234356_BUDI.pdf
- Kumpulkan pada <https://elearning-jti.polije.ac.id/>

h. Kesimpulan

Dengan memahami konsep authorization dalam laravel, menerapkan gates untuk kontrol akses, mengimplementasikan policies dalam laravel, menggunakan middleware authorization dalam routing, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			